Java 入门笔记

第六课 · Java 循环结构与流程转移

学习目标	掌握 while、do-while、for、嵌套循环，以及 break、continue 的作用和常见边界错误。

目录

本课目标 2

3.16 循环的基本组成 2

3.17 while 循环 2

3.18 do-while 循环 3

3.19 for 循环 4

3.20 经典例题：判断素数 5

3.21 循环嵌套 6

3.22 选择结构与循环结构嵌套 6

3.23 break 与 continue 7

常见错误 8

本课小结 8

自测题 8

整理日期：2026 年 10 月 16 日

> 第五课让程序选择分支；本课让程序在条件满足时重复执行，并用 break、continue 控制循环过程。

本课目标

- 理解循环的初始化、条件、循环体和更新四个要素。
- 掌握 while、do-while 和基本 for 循环。
- 会编写嵌套循环，并把选择结构放入循环。
- 正确使用 break、continue，识别死循环和边界错误。

3.16 循环的基本组成

循环通常包含四个部分：

1. 初始化：设置循环变量的初始值。
2. 条件：决定是否继续循环，结果必须为 boolean。
3. 循环体：需要重复执行的代码。
4. 更新：改变循环变量，使循环最终能够结束。

如果条件永远为真，或循环变量没有正确更新，就可能形成死循环。

3.17 while 循环

3.17.1 语法与流程

```
while (条件表达式) {
    循环体;
}
```

```
初始化
  ↓
判断条件？ ——false——> 循环之后
  |
 true
  ↓
执行循环体
  ↓
更新变量
  └————————————————————> 回到条件判断
```

while 先判断条件，再执行循环体，所以循环体可能一次也不执行。

3.17.2 计算 1 到 100 的和

```
int sum = 0;
int i = 1;
while (i <= 100) {
    sum += i;
    i++;
}
System.out.println("1 到 100 的和：" + sum);
```

循环结束时 i 的值为 101，sum 的值为 5050。

3.18 do-while 循环

3.18.1 语法与流程

```
do {
    循环体;
} while (条件表达式);
```

while (□□); 末尾的分号不能省略。

```
初始化
  ↓
执行循环体
  ↓
更新变量
  ↓
判断条件？ ——true——> 回到循环体
  |
 false
  ↓
循环之后
```

do-while 先执行再判断，所以循环体至少执行一次，适合菜单、输入校验等必须先执行一次的场景。

3.18.2 菜单输入示例

```
import java.util.Scanner;

public class MenuDemo {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int choice;
        do {
            System.out.print("请输入 1 查看信息，输入 0 退出：");
            choice = scanner.nextInt();
            if (choice == 1) {
                System.out.println("欢迎学习 Java");
            } else if (choice != 0) {
                System.out.println("无效选项");
            }
        } while (choice != 0);
        scanner.close();
    }
}
```

3.18.3 while 与 do-while

对比项	while	do-while
判断时机	先判断，后执行	先执行，后判断
最少执行次数	0 次	1 次
常见用途	次数未知的重复处理	菜单、至少一次的输入

3.19 for 循环

3.19.1 语法与执行顺序

```
for (初始化表达式; 条件表达式; 更新表达式) {
    循环体;
}
```

```
初始化（仅一次）
      ↓
判断条件？ ——false——> 循环之后
      |
    true
      ↓
执行循环体
      ↓
执行更新表达式
      └————————————————> 回到条件判断
```

for 适合循环次数较明确的场景。

```
int sum = 0;
for (int i = 1; i <= 100; i++) {
    sum += i;
}
System.out.println(sum);
```

3.19.2 省略表达式的合法写法

三个位置都可以省略，但两个分号必须保留，并且要在其他位置完成相应工作。

```
int sum = 0;
int i = 1;
for (; i <= 100; i++) {    // 初始化移到循环前
    sum += i;
}
```

```
int sum = 0;
for (int i = 1; i <= 100;) { // 更新移到循环体
    sum += i;
    i++;
}
```

```
int sum = 0;
for (int i = 1; ; i++) {    // 条件在循环体内判断
    sum += i;
    if (i >= 100) {
        break;
    }
}
```

for (;;) { ... } 表示无限循环，必须有合适的退出方式。for (...); 则是空循环体，通常不是初学者想要的结果。

3.20 经典例题：判断素数

素数是大于 1，并且只能被 1 和自身整除的整数。只需检查到 i * i <= number；如果更大的因数存在，就一定已经有一个与它配对的较小因数。

```
public class PrimeCheck {
    public static void main(String[] args) {
        int number = 29;
        boolean isPrime = number >= 2;

        for (int i = 2; i * i <= number && isPrime; i++) {
            if (number % i == 0) {
                isPrime = false;
            }
        }

        if (isPrime) {
            System.out.println(number + " 是素数");
        } else {
            System.out.println(number + " 不是素数");
        }
    }
}
```

3.21 循环嵌套

循环体内再放一个循环，就形成嵌套循环。外层循环每执行一次，内层循环会完成一轮。

3.21.1 九九乘法表

```
for (int row = 1; row <= 9; row++) {
    for (int column = 1; column <= row; column++) {
        System.out.printf("%d×%d=%-2d ", column, row, column * row);
    }
    System.out.println();
}
```

外层变量 row 控制行数，内层变量 column 控制每行输出多少项。嵌套时要特别检查缩进、大括号和变量名。

3.22 选择结构与循环结构嵌套

3.22.1 输出奇数并统计偶数

```
int evenCount = 0;
for (int i = 1; i <= 10; i++) {
    if (i % 2 != 0) {
        System.out.println(i);
    } else {
        evenCount++;
    }
}
System.out.println("偶数共 " + evenCount + " 个");
```

3.22.2 switch 嵌套在 while 中

```
import java.util.Scanner;

public class SwitchInLoop {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int choice = -1;

        while (choice != 0) {
            System.out.print("请输入 1-3，输入 0 退出：");
            choice = scanner.nextInt();
            switch (choice) {
                case 1:
                case 2:
                case 3:
                    System.out.println("你输入了 " + choice);
                    break;
                case 0:
                    System.out.println("程序退出");
                    break;
                default:
                    System.out.println("输入错误，请输入 0-3");
            }
        }
        scanner.close();
    }
}
```

3.23 break 与 continue

语句	作用	执行后的位置
break	立即结束当前循环或 switch	跳到该结构之后
continue	跳过本次循环的剩余代码	进入下一次迭代
return	结束当前方法	返回调用处

```
for (int i = 1; i <= 10; i++) {
    if (i == 3) {
        continue;        // 不输出 3
    }
    if (i == 8) {
        break;           // 到 8 时结束循环
    }
    System.out.println(i);  // 输出 1、2、4、5、6、7
}
```

在嵌套循环中，普通 break 只结束它所在的最内层循环。Java 支持带标签的 break 和 continue，可控制指定的外层循环，但会增加阅读难度，初学阶段应优先把复杂逻辑拆成方法。

```
outer:
for (int row = 1; row <= 3; row++) {
    for (int column = 1; column <= 3; column++) {
        if (row == 2 && column == 2) {
            break outer;    // 直接结束外层循环
        }
    }
}
```

常见错误

- 把 sum += i 误写成 sum += 1，结果只统计次数而不是累加数值。
- 更新语句漏写分号，或完全忘记更新变量，导致编译错误或死循环。
- 把 sum += i 写成不存在的 sum++i。
- 循环边界混淆 < 与 <=，产生少一次或多一次的错误。
- 在 while 条件或 for 右括号后误写分号，形成空循环体。
- 认为嵌套循环中的普通 break 会同时结束所有循环。

本课小结

- while 先判断，可能执行 0 次；do-while 先执行，至少执行 1 次。
- for 把初始化、条件和更新集中在一行，适合次数明确的循环。
- 嵌套循环常用于二维规律；循环中可嵌套 if 或 switch。
- break 结束当前结构，continue 跳过本次迭代，return 结束方法。
- 下一课学习数组：循环负责重复处理，数组负责批量保存同类型数据。

自测题

1. while 与 do-while 的最少执行次数分别是多少？
2. 写出 for 循环各部分的执行顺序。
3. 普通 break 在两层嵌套循环中会结束哪一层？
4. 编写程序：输出 2 到 100 之间的所有素数，并统计个数。